

Nro. ord.	Apellido y nombre	L.U.	Turno	#hojas
39	ITTIG, Ernesto	685/24	M	3

SISTEMAS DIGITALES - 1C 2025
- 2do Parcial - 19/06/2025

Ej.1	Ej.2	Ej.3	Nota
B	B	B	10

Moteo

Aclaraciones

- Anote apellido, nombre, LU y numere *todas* las hojas entregadas.
- Cada ejercicio será calificado con una de las siguientes tres notas: Bien, Regular o Mal. La división de los ejercicios en incisos es meramente orientativa. Los ejercicios se calificarán globalmente.
- El parcial **no es a libro abierto**, sólo se podrán utilizar las hojas que son entregadas como enunciado.
- Un resultado sin suficiente justificación equivale a un ejercicio no resuelto.
- El parcial se aprueba con al menos dos ejercicios Bien y uno Regular. Para obtener un Regular es necesario demostrar conocimientos sobre el tema del ejercicio.

Ejercicio 1 Implemente la función **rec** en el lenguaje ensamblador RISC V de forma recursiva, respete la convención de llamada presentada en la materia, explique el uso que le dará a cada registro y cómo se asegura que sus valores se preservan antes y después de cada llamada a función. No deben implementar la versión iterativa, debe ser recursiva, en caso contrario se considera desaprobado el ejercicio. Pueden usar la instrucción mul.

$$rec(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ 2 * rec(n - 1) + 3 * rec(n - 2) & \text{si } n > 1 \end{cases}$$

Guía de resolución (opcional):

- Escriba una versión de pseudocódigo.
- Transforme cada caso a su equivalente de operaciones atómicas (descomponga las operaciones lógicas, aritméticas y llamadas a función).
- Identifique los registros a emplear para cada dato.
- Si debe preservar algún registro para respetar la convención, indique qué mecanismo utilizará.
- Defina un flujo de ejecución tentativo.

Importante: Justifique sus respuestas.

Ejercicio 2 Para entender el balance del sistema previsional contamos con un arreglo que contiene una estructura de aportantes con su edad, cantidad de años de aportes y pesos que aportaron al sistema.

Se cuenta con un arreglo **aportantes** que contiene una cantidad fija cuyo tamaño está definido por la constante **largo**. Cada aportante se define como una estructura de cuatro bytes, donde el primer byte determina la **edad** como entero sin signo, el segundo byte determina la cantidad de años de aportes como un entero sin signo y los siguientes dos bytes (half word) determinan la cantidad de pesos que aportaron al sistema.

Escriba un programa que calcule el aporte promedio para las personas cuya edad sea mayor o igual a 65 años.

Ejemplo:

Dirección aportantes	0x0000	0x000001	0x0002	0x0004	0x0005	0x0006
66	43	5000	32	4	-500	
	0x0008	0x000009	0x000A	0x000C	0x000D	0x000E
70	40	-1000	25	5	200	

Esqueleto de programa:

Importante: Justifique sus respuestas.

Address	Instruction	Type	Fields							Machine Language
			imm _{12,10:5}	rs2	rs1	f3	imm _{4,1:11}	op		
0x100C	beq x4, x4, L7	B	1111111	00100	00100	000	10101	1100011	FE420AE3	

Ejercicio 1:

Algoritmo para rec en PSEUDO-C:

```
int rec(int n) {
    if (n <= 1)
        return n;
    int resultado = rec(n-1);
    resultado = 2 * resultado;
    int tmp = rec(n-2);
    tmp = 3 * tmp;
    return resultado + tmp;
}
```

L_return_0 ← 0

Voy a usar s0 para n, y s1 para resultado.
Como son registros permanentes, los guardaré en el stack en el ~~prologo~~ y luego los restauraré en el epilogo.

```
12(sp) ← ra
8(sp) ← s0
4(sp) ← s1
```

Resolución en ensamblador RISC-V:

rec:

```
addi sp, sp, -16
sw ra, 12(sp)
sw s0, 8(sp)
sw s1, 4(sp)
```

Fin ~~prologo~~ prologo

```
li t0, 1
```

```
bgt a0, t0, L_else_0
```

```
L_return_0:
```

L_else_0:

~~addi~~

```
mv s0, a0
```

```
addi a0, s0, -1
```

```
jal rec
```

li t0, 2 ← mul s1, a0, t0

```
addi a0, s0, -2
```

```
jal rec
```

```
li t0, 3
```

```
mul a0, a0, t0
```

```
add a0, a0, s1
```

L_return_0:

```
lw s1, 4(sp)
```

```
lw s0, 8(sp)
```

```
lw ra, 12(sp)
```

```
addi sp, sp, 16
```

```
ret
```

if (n <= 1)
return n;

s0 es n

resultado = 2 * rec(n-1)

a0 = 3 * rec(n-2)
return a0 + resultado

Ejercicio 2:

struct Aportante { uint8_t edad; uint8_t años; int16_t aportes; }
 Se accede con: 0(reg) 1(reg) 2 (reg)
 Se incrementa de a 4.

Algoritmo en PSEUDO-C:

```
Aportante * arr; int largo; int suma = 0; n = 0;
while (largo > 0) {
  if (arr->edad >= 65) {
    suma += arr->aportes;
    n += 1;
  }
  arr += 1; // le sumo 4 bytes
  largo -= 1;
}
resultado = suma / n;
```

```
a0 ← arr
a1 ← largo
t0 ← suma
t1 ← n
```

Resolución en RISC-V:

- data:

aportantes:

etc...

largo : .byte 4

.text:

la a0, aportantes

la a1, largo

li t0, a0
li t1, a1

lbu a1, 0(a1)

while_0:

beqz a1, end_while_0

li t2, 65

~~li t3, 0~~

lbu t3, 0(a0)

blt t3, t2, else_0

lh t3, 2(a0)

add t0, t0, t3

addi t1, t1, 1

else_0:

addi a0, 4

addi a1, -1

j while_0

end_while_0:

div a0, t0, t1

fin:

j fin

arr → a0

largo → a1

while (largo > 0)

if (arr->edad >= 65)

suma += arr->aportes;

n += 1

arr += 1 (cada Aportante son 4 bytes)

largo -= 1

resultado = suma / n; el resultado queda en a0

halt

